

LAB 02

Stored Procedures & Cursors

Writing reusable SQL logic, and processing result sets one row at a time

Today's Session

A 90-minute walkthrough, from syntax to hands-on examples

1 Stored procedures: what & why

2 Syntax, parameters & nesting

3 Hands-on procedure examples

4 The employee / project schema

5 What is a cursor?

6 Cursor lifecycle & syntax

7 Hands-on cursor examples

8 Cursors vs. set-based SQL

What Is a Stored Procedure?

A block of SQL, saved under a name, that you call instead of retyping.

R

Reusable — Write the logic once, then call it from anywhere, as often as needed.

P

Parameterized — Pass values in so one procedure adapts to different inputs and callers.

P

Precompiled — The server builds an execution plan once and caches it for faster runs.

C

Centralized — Business logic lives in one place instead of scattered across every app.

Defining, Calling, Dropping

```
DELIMITER $$

CREATE PROCEDURE procedure_name(parameters)
BEGIN
    SQL statements;
END $$

DELIMITER ;

-- Run it
CALL procedure_name();

-- Remove it
DROP PROCEDURE procedure_name;
```

- DELIMITER swaps the statement terminator so semicolons inside the body don't end the CREATE statement early
- CALL executes the saved procedure
- DROP PROCEDURE removes it permanently

Three Kinds of Parameters

Parameters decide how values flow between the caller and the procedure.



IN

Default parameter. Passes a value into the procedure; read-only inside the body.



OUT

Sends a value back to the calling program once the procedure finishes.



INOUT

Both directions at once — receives a value, and can return a changed one.

A First Example

```
DELIMITER //
```

```
CREATE PROCEDURE SelectAllCustomers()  
BEGIN  
    SELECT * FROM Customers;  
END //
```

```
DELIMITER ;
```

```
CALL SelectAllCustomers();
```

No parameters here — the same result set comes back every time it's called.

Adding Parameters

```
DELIMITER //  
  
CREATE PROCEDURE SelectAllCustomers(  
    IN input VARCHAR(30),  
    IN code INT  
)  
BEGIN  
    SELECT *  
    FROM Customers  
    WHERE City = input  
        AND PostalCode = code;  
END //  
  
DELIMITER ;  
  
CALL SelectAllCustomers('London', 12345);
```

Two IN parameters shape the WHERE clause at call time — same procedure, different results per caller.

Nested Stored Procedures

- Calling procedure X inside the body of procedure Y is called “nesting”
- Breaks large blocks of SQL into smaller, reusable pieces
- Improves modularity — fix or extend one piece without touching the rest

```
DELIMITER //
```

```
CREATE PROCEDURE say_hello(IN name VARCHAR(50))  
BEGIN  
    SELECT CONCAT('Hello, ', name, '!')  
    AS greeting;  
END //
```

```
CREATE PROCEDURE greet_team()  
BEGIN  
    CALL say_hello('Alice');  
    CALL say_hello('Bob');  
END //
```

```
DELIMITER ;  
CALL greet_team();
```


PART TWO

Hands-On: The Company Schema

Four tables we'll reuse for every example ahead

The Practice Schema

Departments

dept_id · dept_name · location

Employees

emp_id · emp_name · dept_id · manager_id ·
salary · hire_date

Projects

project_id · project_name · budget

Project_Assignments

emp_id · project_id · hours_worked

Employees self-references manager_id — every manager is also an employee.

Inserting Safely: AddEmployee

```
CREATE PROCEDURE AddEmployee (  
    IN p_id INT, IN p_name VARCHAR(100),  
    IN p_dept INT, IN p_manager INT,  
    IN p_salary DECIMAL(10,2), IN p_hire DATE  
)  
BEGIN  
    INSERT INTO Employees  
        (emp_id, emp_name, dept_id,  
         manager_id, salary, hire_date)  
    VALUES  
        (p_id, p_name, p_dept,  
         p_manager, p_salary, p_hire);  
END //
```

Six IN parameters — one call
inserts one clean, validated row.

Updating & Reporting

```
CREATE PROCEDURE UpdateSalary (  
    IN p_emp_id INT,  
    IN p_new_salary DECIMAL(10,2)  
)  
BEGIN  
    UPDATE Employees  
    SET salary = p_new_salary  
    WHERE emp_id = p_emp_id;  
END //
```

```
CREATE PROCEDURE GetDeptStats (  
    IN p_dept_id INT,  
    OUT p_emp_count INT,  
    OUT p_total_salary DECIMAL(12,2)  
)  
BEGIN  
    SELECT COUNT(*),  
           COALESCE(SUM(salary),0)  
    INTO p_emp_count, p_total_salary  
    FROM Employees  
    WHERE dept_id = p_dept_id;  
END //
```

GetDeptStats uses OUT parameters — call it, then read the values it hands back to you.

Conditional Logic: ApplyTenureBonus

```
CREATE PROCEDURE ApplyTenureBonus (  
    IN p_dept_id INT,  
    IN p_bonus_amount DECIMAL(8,2)  
)  
BEGIN  
    UPDATE Employees  
    SET salary = salary + p_bonus_amount  
    WHERE dept_id = p_dept_id  
        AND hire_date < '2020-01-01';  
END //
```

The WHERE clause is the condition — only pre-2020 hires in the given department get the bonus.

PART THREE

Cursors

Processing a result set one row at a time

What Is a Cursor?

A temporary workspace the server allocates so you can step through a result set row by row.

Implicit

Created automatically by the server for every DML statement — you never declare it yourself.

Explicit

Declared by you, when you need a row-by-row FETCH loop over a chosen query.

Should You Reach for a Cursor?

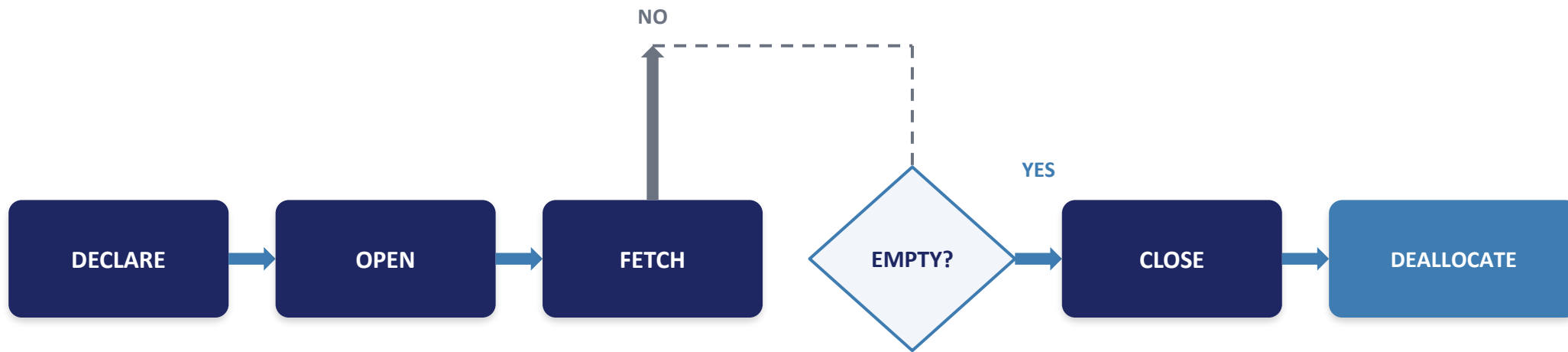
Relational databases are built to handle data in sets — a single UPDATE or JOIN usually outperforms a row-by-row loop.

Use when — logic genuinely depends on one row at a time — e.g., calling an external routine per row

Avoid when — a JOIN, UPDATE ... WHERE, or aggregate query can express the same result

Remember — cursors can very often be replaced by JOINS

Life Cycle of a Cursor



A CONTINUE HANDLER sets a done flag when FETCH runs out of rows — that flag drives the EMPTY? check.

Cursor Syntax

```
DECLARE variables, records;
DECLARE cursor_name CURSOR FOR select_statement;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

OPEN cursor_name;
read_loop: LOOP
    FETCH cursor_name INTO variables;
    IF done THEN LEAVE read_loop; END IF;
    -- process the row
END LOOP;
CLOSE cursor_name;
DEALLOCATE cursor_name;
```

Row-by-Row Reporting

```
CREATE PROCEDURE CursorPrintAllEmployees()
BEGIN
    DECLARE v_done INT DEFAULT 0;
    DECLARE v_name VARCHAR(100);
    DECLARE v_sal DECIMAL(10,2);
    DECLARE cur CURSOR FOR
        SELECT emp_name, salary FROM Employees;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_name, v_sal;
        IF v_done THEN LEAVE read_loop; END IF;
        SELECT CONCAT('Employee: ', v_name, ' | Salary: ', v_sal);
    END LOOP;
    CLOSE cur;
END //
```

Prints one formatted line per employee — a good first cursor to trace by hand.

Row-by-Row Updates

```
CREATE PROCEDURE CursorInflationAdjustment()  
BEGIN  
    DECLARE v_done INT DEFAULT 0;  
    DECLARE v_id INT;  
    DECLARE v_sal DECIMAL(10,2);  
    DECLARE cur CURSOR FOR  
        SELECT emp_id, salary FROM Employees  
        WHERE salary < 80000;  
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;  
  
    OPEN cur;  
    read_loop: LOOP  
        FETCH cur INTO v_id, v_sal;  
        IF v_done THEN LEAVE read_loop; END IF;  
        UPDATE Employees SET salary = v_sal * 1.10  
        WHERE emp_id = v_id;  
    END LOOP;  
    CLOSE cur;  
END //
```

Discuss: could a single UPDATE ... WHERE salary < 80000 replace this entire cursor?

Conditional Alerts, Row by Row

```
CREATE PROCEDURE CursorLogHighBudgetProjects()
BEGIN
    DECLARE v_done INT DEFAULT 0;
    DECLARE v_pname VARCHAR(100);
    DECLARE v_budget DECIMAL(12,2);
    DECLARE cur CURSOR FOR
        SELECT project_name, budget FROM Projects;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_pname, v_budget;
        IF v_done THEN LEAVE read_loop; END IF;
        IF v_budget > 100000 THEN
            SELECT CONCAT('High Priority: ', v_pname);
        END IF;
    END LOOP;
    CLOSE cur;
END //
```

Combines a cursor loop with an IF check inside it — logic per row, not just data per row.

Auditing Assignments

```
CREATE PROCEDURE CursorAuditAssignments()
BEGIN
    DECLARE v_done INT DEFAULT 0;
    DECLARE v_emp INT;
    DECLARE v_proj INT;
    DECLARE v_hours INT;
    DECLARE cur CURSOR FOR
        SELECT emp_id, project_id, hours_worked
        FROM Project_Assignments;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur;
read_loop: LOOP
    FETCH cur INTO v_emp, v_proj, v_hours;
    IF v_done THEN LEAVE read_loop; END IF;
    IF v_hours > 150 THEN
        SELECT CONCAT('Heavy workload: Emp ', v_emp);
    END IF;
END LOOP;
CLOSE cur;
END //
```

A row-by-row audit check — flags any assignment over 150 logged hours.

Try It Yourself

Using the schema from Part Two, work through these on your own machine.

- 1 Write a procedure `GetEmployeesByDept(IN dept_id)` that selects every employee in a department
- 2 Add an OUT parameter that also returns the department's average salary
- 3 Write a cursor that gives a 5% raise to every employee on the Cloud Migration project
- 4 Rewrite one cursor example as a single set-based UPDATE or JOIN, and compare the results

Key Takeaways

- Stored procedures package reusable SQL and accept IN / OUT / INOUT parameters
- Nesting procedures keeps large logic modular and easier to maintain
- Cursors step through results row by row — powerful, but not always necessary
- Reach for a JOIN or a set-based statement before reaching for a cursor

References

- [geeksforgeeks.org](https://www.geeksforgeeks.org/difference-between-functions-and-stored-procedures-in-plsql/) — Difference Between Functions and Stored Procedures in PL/SQL
- [docs.oracle.com](https://docs.oracle.com/en/database/oracle/oracle-database/19/plsql/overview.html) — PL/SQL Overview
- [mysqltutorial.org](https://www.mysqltutorial.org/stored-procedure-parameters/) — Stored Procedure Parameters
- [learn.microsoft.com](https://learn.microsoft.com/en-us/sql/t-sql/statements/declare-cursor-transact-sql?view=sql-server-2017) — DECLARE CURSOR (Transact-SQL)
- [coderpad.io](https://coderpad.io/advanced-stored-procedures-in-mysql/) — Advanced Stored Procedures in MySQL